PYTHON PROGRAMMING | LAB RECORD

Lab Experiments

Using Unit 4 Concepts - File Handling & Data Processing

NAME	**Sriram Satvik Karanam**
USN	**25BTRAD049**
BRANCH	**CSE - AIDE (B Section)**
COURSE	**Python Programming**
PROGRAMS	**Lab Exercises 11, 12 and 13**

Contents

Lab	Program	Main file concept	Skills assessed
11	Log File Reader Program	Reading text files line by line, keyword counting	CT + RWE
12	Student Data Manager Using Files	Append / read / rewrite - full record management (CRUD)	CT + RWE
13	CSV-Based Payroll Management System	csv module, calculation logic, summaries	CT + RWE

CT = Critical Thinking (analysis, design, validation, verification) | **RWE** = Real World Exposure (application to a real scenario). Every program below was actually executed; the outputs printed in this document are the real outputs.

Each lab section follows the same structure: **Question** → **Aim** → **Analysis & Design (CT)** → **Complete Code** → **Sample Data** → **Output** → **Testing & Verification (CT)** → **Real-World Application (RWE)** → **Conclusion.**

LAB EXERCISE 11

Log File Reader Program

Read a log file • detect ERROR / WARNING / INFO • summarise occurrences

■ QUESTION - Lab Exercise 11

Log File Reader Program: Develop a Python program to **read a log file**, identify specific keywords such as **ERROR, WARNING and INFO**, and **summarise the number of occurrences** of each category.

(CT) Analyse log contents, design suitable **search and counting logic**, **handle missing or empty files**, and **verify the accuracy** of results.

(RWE) Apply the program to a **sample application log** and explain how it helps **identify errors and monitor system activity**.

1. Aim

To write a Python program that reads an application log file, classifies every entry as **ERROR**, **WARNING** or **INFO**, counts each category, reports percentages, lists the failing lines and finally gives an overall health status of the system.

2. Concepts Used

Concept	Where it is used in the program
File handling - `open()` in read mode, `with` statement	The log is opened with `with open(path, 'r')` so it is closed automatically, even if an error occurs.
Iterating over a file	`for number, line in enumerate(log, start=1)` reads one line at a time - works even for very large logs.
Regular expressions (`re`)	`\b(ERROR|WARNING|INFO)\b` with `IGNORECASE` finds the keyword as a whole word.
`collections.Counter` / dictionary	Stores the number of occurrences of each level.
Exception handling	`FileNotFoundError`, `ValueError` (empty file), `PermissionError`.
Functions & modular design	`analyze_log()`, `print_report()`, `top_repeated_errors()`, `run_tests()`.

3. Critical Thinking - Analysis and Design

3.1 Analysing the log contents

A typical application log line has the form `DATE TIME LEVEL message`. The **level** is a single word that tells how serious the event is:

- **INFO** - normal activity (server started, user logged in, backup completed).
- **WARNING** - something unusual that may become a problem (disk 82% full, slow query).
- **ERROR** - an operation has failed (database timeout, file not found).

3.2 Search logic

- Compile one regular expression once (`LEVEL_PATTERN`) and reuse it for every line - faster than searching three times.
- **Word boundaries (`\b`)** ensure that a word such as *INFORMATION* is not counted as INFO.
- **Case-insensitive** so *error:*, *Error* and *ERROR* are all recognised.
- Only the **first** keyword in a line is used, so a line is never counted in two categories.

3.3 Counting logic - algorithm

Step	Action
1	Check the file exists; if not, report *file not found* and stop.
2	Check the file size; if it is 0 bytes, report *file is empty* and stop.
3	Initialise counts: ERROR = WARNING = INFO = 0, total = 0, unclassified = 0.
4	Read the file line by line; skip blank lines; increase *total*.
5	Search the line for a keyword. If found, increase that level's count (and remember ERROR / WARNING lines with their line numbers); otherwise increase *unclassified*.
6	If the file contained only blank lines, report an error; otherwise calculate percentages, repeated errors and status, then print the report.

3.4 Edge cases handled

Situation	How the program reacts
File does not exist	`FileNotFoundError` is raised and a clear message is printed - no crash.
File is empty (0 bytes) or has only blank lines	`ValueError` with the message *Log file is empty*.
Line without any keyword	Counted under **OTHER** so that ERROR + WARNING + INFO + OTHER always equals the total.
Unreadable characters in the log	`errors='replace'` prevents a UnicodeDecodeError.
Percentage calculation	Division is safe because an empty file is rejected before dividing by *total*.

4. Complete Program - log_analyzer.py

```python
"""
Lab Exercise 11 : Log File Reader Program
Reads a log file, counts ERROR / WARNING / INFO entries and summarises them.
Usage : python log_analyzer.py [path_to_log]      |  python log_analyzer.py --test
"""
import os
import re
import sys
import tempfile
from collections import Counter

LEVELS = ("ERROR", "WARNING", "INFO")
# \b = word boundary, so "ERRORS" or "INFORMATION" are NOT mis-counted
LEVEL_PATTERN = re.compile(r"\b(ERROR|WARNING|INFO)\b", re.IGNORECASE)


def analyze_log(path):
    """Scan the log once; return (counts, flagged_lines, total_lines, unclassified)."""
    if not os.path.exists(path):
        raise FileNotFoundError(f"Log file not found: {path}")
    if os.path.getsize(path) == 0:
        raise ValueError(f"Log file is empty: {path}")

    counts = Counter({level: 0 for level in LEVELS})
    flagged = {"ERROR": [], "WARNING": []}
    total = unclassified = 0

    with open(path, "r", encoding="utf-8", errors="replace") as log:
        for number, line in enumerate(log, start=1):
            line = line.strip()
            if not line:                    # ignore blank lines
                continue
            total += 1
            match = LEVEL_PATTERN.search(line)  # first keyword decides the level
            if match is None:
                unclassified += 1
                continue
            level = match.group(1).upper()
            counts[level] += 1
            if level in flagged:
                flagged[level].append((number, line))

    if total == 0:
        raise ValueError(f"Log file contains only blank lines: {path}")
    return counts, flagged, total, unclassified


def top_repeated_errors(error_lines, n=3):
    """Group identical error messages (timestamp removed) to spot recurring faults."""
    messages = Counter()
    for _, line in error_lines:
        text = re.split(r"\bERROR\b", line, maxsplit=1, flags=re.IGNORECASE)[-1]
        messages[text.strip(" :-")] += 1
    return messages.most_common(n)


def print_report(path, counts, flagged, total, unclassified):
    print("=" * 62)
    print(f" LOG ANALYSIS REPORT  -  {os.path.basename(path)}")
    print("=" * 62)
    print(f" Total log entries : {total}")
    print("-" * 62)
    print(f" {'LEVEL':<10}{'COUNT':>7}{'SHARE':>9}   GRAPH")
    for level in LEVELS:
        share = counts[level] * 100 / total
        bar = "#" * round(share / 4)
        print(f" {level:<10}{counts[level]:>7}{share:>8.1f}%   {bar}")
    if unclassified:
        print(f" {'OTHER':<10}{unclassified:>7}{unclassified*100/total:>8.1f}%")
    print("-" * 62)

    if flagged["ERROR"]:
        print(" ERROR lines (line no : message)")
        for number, line in flagged["ERROR"]:
            print(f"   [{number:>3}] {line}")
        print(" Most frequent errors:")
        for message, times in top_repeated_errors(flagged["ERROR"]):
            print(f"   {times} x {message}")
    ratio = counts["ERROR"] * 100 / total
    status = "CRITICAL - investigate now" if ratio >= 20 else \
             "ATTENTION - review errors" if counts["ERROR"] else "HEALTHY"
    print("-" * 62)
```

```
    print(f" System status : {status}  (error ratio {ratio:.1f}%)")
    print("=" * 62)


def run_tests():
    """Verify the counting logic against a log whose answer is known in advance."""
    sample = ("2026-01-01 INFO started\n2026-01-01 ERROR disk full\n\n"
              "2026-01-01 WARNING low memory\n2026-01-01 error: timeout\n"
              "no keyword here\n2026-01-01 INFORMATION only (no keyword)\n")
    with tempfile.TemporaryDirectory() as folder:
        good = os.path.join(folder, "t.log")
        with open(good, "w") as f:
            f.write(sample)
        counts, _, total, other = analyze_log(good)
        assert (counts["ERROR"], counts["WARNING"], counts["INFO"]) == (2, 1, 1)
        assert (total, other) == (6, 2)
        print("TEST 1 passed : counts ERROR=2 WARNING=1 INFO=1, total=6, other=2")

        empty = os.path.join(folder, "e.log")
        open(empty, "w").close()
        try:
            analyze_log(empty)
        except ValueError as err:
            print("TEST 2 passed : empty file ->", err.__class__.__name__)
        try:
            analyze_log(os.path.join(folder, "nope.log"))
        except FileNotFoundError as err:
            print("TEST 3 passed : missing file ->", err.__class__.__name__)

        # independent cross-check using a different method (substring search)
        alt = sum(1 for l in sample.splitlines() if "ERROR" in l.upper()
                  and "INFORMATION" not in l.upper())
        assert alt == counts["ERROR"]
        print("TEST 4 passed : result matches independent cross-check")


def main():
    if "--test" in sys.argv:
        run_tests()
        return
    path = sys.argv[1] if len(sys.argv) > 1 else input("Enter log file path: ").strip()
    try:
        print_report(path, *analyze_log(path))
    except FileNotFoundError as err:
        print("[ERROR]", err)
    except ValueError as err:
        print("[WARNING]", err)
    except PermissionError:
        print("[ERROR] Permission denied while reading", path)


if __name__ == "__main__":
    main()
```

5. Sample Application Log - app.log

A 20-line log of a college web application (student portal) is used as the input. Keywords are colour-coded for readability.

```
2026-10-20 09:00:01 INFO  Application server started on port 8080
2026-10-20 09:00:03 INFO  Connected to database college_db
2026-10-20 09:01:15 INFO  User admin logged in
2026-10-20 09:02:42 WARNING Disk usage at 82% on /var/data
2026-10-20 09:04:10 INFO  Student record 25BTRAD049 fetched
2026-10-20 09:05:27 ERROR Database connection timeout after 30s
2026-10-20 09:05:40 INFO  Retrying database connection
2026-10-20 09:05:48 INFO  Database connection restored
2026-10-20 09:10:09 WARNING Slow query detected (4.2s) on table marks
2026-10-20 09:12:55 INFO  Report generated for semester 3
2026-10-20 09:20:31 ERROR File not found: /uploads/timetable.pdf
2026-10-20 09:25:00 WARNING Memory usage above 75%
2026-10-20 09:31:14 INFO  User faculty01 logged in
2026-10-20 09:40:48 ERROR Database connection timeout after 30s
2026-10-20 09:41:02 INFO  Retrying database connection
2026-10-20 09:45:10 INFO  Backup job completed successfully
2026-10-20 09:50:22 ERROR Payment gateway unreachable
2026-10-20 09:55:37 WARNING Unusual login attempts from 10.0.0.45
2026-10-20 10:00:00 INFO  Scheduled health check passed
2026-10-20 10:05:12 ERROR Database connection timeout after 30s
```

6. Output

6.1 Run: `python log_analyzer.py app.log`

```
============================================================
 LOG ANALYSIS REPORT  -  app.log
============================================================
 Total log entries : 20
------------------------------------------------------------
 LEVEL       COUNT    SHARE   GRAPH
 ERROR           5    25.0%   ######
 WARNING         4    20.0%   #####
 INFO           11    55.0%   ##############
------------------------------------------------------------
 ERROR lines (line no : message)
   [  6] 2026-10-20 09:05:27 ERROR Database connection timeout after 30s
   [ 11] 2026-10-20 09:20:31 ERROR File not found: /uploads/timetable.pdf
   [ 14] 2026-10-20 09:40:48 ERROR Database connection timeout after 30s
   [ 17] 2026-10-20 09:50:22 ERROR Payment gateway unreachable
   [ 20] 2026-10-20 10:05:12 ERROR Database connection timeout after 30s
 Most frequent errors:
   3 x Database connection timeout after 30s
   1 x File not found: /uploads/timetable.pdf
   1 x Payment gateway unreachable
------------------------------------------------------------
 System status : CRITICAL - investigate now  (error ratio 25.0%)
============================================================
```

6.2 Handling a missing file and an empty file

`python log_analyzer.py missing.log` and `python log_analyzer.py empty.log`

```
[ERROR] Log file not found: missing.log
```

```
[WARNING] Log file is empty: empty.log
```

7. Testing and Verification (CT)

Manual verification: reading app.log line by line gives ERROR on lines 6, 11, 14, 17, 20 (**5**); WARNING on lines 4, 9, 12, 18 (**4**); the remaining **11** lines are INFO. 5 + 4 + 11 = 20 = total lines, so the program output matches the manual count exactly.

Test	Input	Expected	Result
1	Known 6-line sample with a lower-case *error:*, a blank line and the word INFORMATION	ERROR 2, WARNING 1, INFO 1, total 6, other 2	**Pass**
2	Empty file	ValueError - file is empty	**Pass**
3	Non-existent file	FileNotFoundError	**Pass**
4	Cross-check by a second method (substring search)	Same ERROR count as the regex method	**Pass**

Automatic test run - `python log_analyzer.py --test`

```
TEST 1 passed : counts ERROR=2 WARNING=1 INFO=1, total=6, other=2
TEST 2 passed : empty file -> ValueError
TEST 3 passed : missing file -> FileNotFoundError
TEST 4 passed : result matches independent cross-check
```

8. Real World Exposure (RWE) - how this helps

Interpreting the sample log. The report shows that **25% of all entries are ERRORs**, which crosses the program's 20% alert limit, so the status is CRITICAL. The most important finding is that **the same error - *Database connection timeout after 30s* - occurred 3 times** (lines 6, 14 and 20). A repeating error points to a root cause (for example an overloaded database or a small connection pool) rather than a one-time glitch. The WARNING lines (disk at 82%, memory above 75%, slow query, unusual login attempts) are early signs that were visible *before* the failures became

serious.

Purpose in a real system	How this program supports it
Identify errors quickly	Lists every ERROR with its line number, so an engineer jumps directly to the faulty event instead of reading thousands of lines.
Find recurring faults	The 'most frequent errors' section groups identical messages and exposes patterns.
Monitor system activity	INFO count shows normal workload (logins, jobs, reports); a sudden drop or rise indicates a change in system behaviour.
Early warning / preventive action	WARNING entries (disk, memory, slow queries) allow action before a crash happens.
Security	Many repeated login warnings may indicate a brute-force attempt (line 18 of the sample log).
Automation	The script can be scheduled (e.g. every hour) so that the health status is checked without human effort. Tools such as ELK, Splunk and CloudWatch work on the same principle at a much larger scale.

Conclusion: The program reads the log in a single pass, classifies and counts ERROR, WARNING and INFO entries accurately, handles missing / empty files gracefully, and converts raw text into a readable summary that supports fast troubleshooting and continuous monitoring.

LAB EXERCISE 12

Student Data Manager Using Files

Store • retrieve • search • update • display student records using file operations

■ QUESTION - Lab Exercise 12

Student Data Manager Using Files: Develop a Python program to **store, retrieve, search, update, and display** student records using **file operations**.

(CT) Design appropriate **record-management logic**, select suitable **file operations**, **validate inputs**, and **test data retrieval and updates**.

(RWE) Apply the program to a **college student-record management scenario** and explain its usefulness in maintaining student information.

1. Aim

To build a menu-driven Python application that keeps student records permanently in a text file and supports the complete set of record operations: **add (store), view (retrieve), search, update, delete, display** and simple statistics - with strict validation of every input.

2. Critical Thinking - Record Management Design

2.1 Record structure (file format)

Each student is one line in `students.txt`; fields are separated by the pipe character `|` (a character that does not occur in names, so splitting is safe). Example line: `25BTRAD049|Sriram Satvik Karanam|CSE-AIDE|3|8.9`

Field	Meaning	Validation rule applied
USN	Unique student identifier (primary key)	Pattern: 2 digits + 3-6 capital letters + 3 digits (e.g. 25BTRAD049); converted to upper case; no duplicates.
Name	Student name	2-50 characters; letters, space, . ' - only; stored in Title Case.
Branch	Department	2-20 letters (e.g. CSE-AIDE); stored in upper case.
Semester	Current semester	Whole number from 1 to 8.
CGPA	Cumulative grade point	Number between 0 and 10; rounded to 2 decimals.

2.2 Choosing the right file operation

Operation	Function	File mode / technique	Reason for the choice
Store (add)	add_student()	`'a'` (append)	Adds one line at the end without touching existing data - fast and safe.
Retrieve / display	load_all(), display()	`'r'` (read)	Reads every line, splits at '|', builds a list of dictionaries; a missing file simply means 'no records yet'.
Search	search_students()	Read + filter	Exact USN match, or partial (case-insensitive) match on name / exact branch.
Update	update_student()	Read all → modify → `'w'` rewrite	Text files cannot be edited in the middle, so the whole file is rewritten.
Delete	delete_student()	Read all → filter → rewrite	Same technique; the record is dropped from the list.

Operation	Function	File mode / technique	Reason for the choice
Safe saving	save_all()	Write to `.tmp` file, then `os.replace()`	If the program stops while writing, the original file is not corrupted.

2.3 Other design decisions

- **Validation before storage:** each field has its own validator that raises `ValueError` with a clear message; the menu re-asks until the input is correct (`ask()` helper).
- **Duplicate prevention:** USN acts as a primary key - a second record with the same USN is refused.
- **Robust loading:** blank or corrupted lines (wrong number of fields) are skipped instead of crashing the program.
- **USN cannot be updated** because it identifies the record; all other fields can be changed.
- **Separation of logic and interface:** operations are plain functions (testable), the menu is a thin layer on top.

3. Complete Program - student_manager.py

```python
"""
Lab Exercise 12 : Student Data Manager Using Files
Store, retrieve, search, update, delete and display student records in a text file.
Record format (one student per line, fields separated by '|'):
    USN|Name|Branch|Semester|CGPA
Usage : python student_manager.py          (interactive menu)
        python student_manager.py --demo   (scripted demonstration / tests)
"""
import os
import re
import sys

DATA_FILE = "students.txt"
DELIM = "|"
FIELDS = ("usn", "name", "branch", "semester", "cgpa")
USN_RE = re.compile(r"^\d{2}[A-Z]{3,6}\d{3}$")          # e.g. 25BTRAD049
NAME_RE = re.compile(r"^[A-Za-z][A-Za-z .'-]{1,49}$")


# ---------------------------------------------------------------- validation
def validate_usn(value):
    value = value.strip().upper()
    if not USN_RE.match(value):
        raise ValueError("USN must look like 25BTRAD049 (2 digits + letters + 3 digits)")
    return value

def validate_name(value):
    value = " ".join(value.split())
    if not NAME_RE.match(value):
        raise ValueError("Name must be 2-50 characters (letters, spaces, . ' - only)")
    return value.title()

def validate_branch(value):
    value = value.strip().upper()
    if not re.match(r"^[A-Z][A-Z -]{1,19}$", value):
        raise ValueError("Branch must be 2-20 letters, e.g. CSE-AIDE")
    return value

def validate_semester(value):
    if not str(value).strip().isdigit() or not 1 <= int(value) <= 8:
        raise ValueError("Semester must be a whole number from 1 to 8")
    return int(value)

def validate_cgpa(value):
    try:
        number = float(value)
    except ValueError:
        raise ValueError("CGPA must be a number") from None
    if not 0.0 <= number <= 10.0:
        raise ValueError("CGPA must be between 0 and 10")
    return round(number, 2)

VALIDATORS = {"usn": validate_usn, "name": validate_name, "branch": validate_branch,
              "semester": validate_semester, "cgpa": validate_cgpa}


# ---------------------------------------------------------------- file handling
def load_all():
    """Read every valid record from the file (a missing file means no records yet)."""
    records = []
    if not os.path.exists(DATA_FILE):
        return records
    with open(DATA_FILE, "r", encoding="utf-8") as f:
        for line in f:
            parts = line.rstrip("\n").split(DELIM)
            if len(parts) != len(FIELDS):
                continue                        # skip blank / corrupted lines
            records.append(dict(zip(FIELDS, parts)))
    return records

def save_all(records):
    """Write to a temporary file first, then replace - protects against half-written data."""
    temp = DATA_FILE + ".tmp"
    with open(temp, "w", encoding="utf-8") as f:
        for r in records:
            f.write(DELIM.join(str(r[k]) for k in FIELDS) + "\n")
    os.replace(temp, DATA_FILE)


# ---------------------------------------------------------------- operations
def add_student(usn, name, branch, semester, cgpa):
    record = {"usn": validate_usn(usn), "name": validate_name(name),
```

```
             "branch": validate_branch(branch), "semester": validate_semester(semester),
             "cgpa": validate_cgpa(cgpa)}
    records = load_all()
    if any(r["usn"] == record["usn"] for r in records):
        raise ValueError(f"Duplicate USN {record['usn']} - record already exists")
    with open(DATA_FILE, "a", encoding="utf-8") as f:          # append mode
        f.write(DELIM.join(str(record[k]) for k in FIELDS) + "\n")
    return record

def search_students(keyword):
    """Exact USN match or case-insensitive partial name / branch match."""
    key = keyword.strip().lower()
    return [r for r in load_all()
            if key == r["usn"].lower() or key in r["name"].lower() or key == r["branch"].lower()]

def update_student(usn, **changes):
    usn = validate_usn(usn)
    records = load_all()
    for r in records:
        if r["usn"] == usn:
            for field, value in changes.items():
                if field == "usn" or field not in VALIDATORS:
                    raise ValueError(f"Field '{field}' cannot be updated")
                r[field] = VALIDATORS[field](value)
            save_all(records)
            return r
    raise KeyError(f"No student found with USN {usn}")

def delete_student(usn):
    usn = validate_usn(usn)
    records = load_all()
    kept = [r for r in records if r["usn"] != usn]
    if len(kept) == len(records):
        raise KeyError(f"No student found with USN {usn}")
    save_all(kept)

def display(records):
    if not records:
        print("  (no records to show)")
        return
    line = "+-------------+----------------------+------------+-----+-------+"
    print(line)
    print(f"| {'USN':<11} | {'NAME':<21} | {'BRANCH':<10} | {'SEM':<3} | {'CGPA':<5} |")
    print(line)
    for r in sorted(records, key=lambda x: x["usn"]):
        print(f"| {r['usn']:<11} | {r['name']:<21} | {r['branch']:<10} | "
              f"{r['semester']:<3} | {float(r['cgpa']):<5.2f} |")
    print(line)
    print(f"  {len(records)} record(s)")

def statistics():
    records = load_all()
    if not records:
        print("  No data for statistics.")
        return
    cgpas = [float(r["cgpa"]) for r in records]
    top = max(records, key=lambda r: float(r["cgpa"]))
    print(f"  Students: {len(records)} | Average CGPA: {sum(cgpas)/len(cgpas):.2f} | "
          f"Topper: {top['name']} ({top['cgpa']})")


# ---------------------------------------------------------------- user interface
def ask(prompt, validator):
    """Keep asking until the input passes validation."""
    while True:
        try:
            return validator(input(prompt))
        except ValueError as err:
            print("  Invalid input:", err)

def menu():
    options = """
 ===== STUDENT DATA MANAGER =====
  1. Add student      4. Update student
  2. View all         5. Delete student
  3. Search           6. Statistics
  0. Exit"""
    while True:
        print(options)
        choice = input(" Choose an option: ").strip()
        try:
            if choice == "1":
                rec = add_student(ask(" USN      : ", validate_usn), ask(" Name     : ", validate_name),
                                  ask(" Branch   : ", validate_branch), ask(" Semester : ", validate_semester),
                                  ask(" CGPA     : ", validate_cgpa))
                print("  Saved:", rec["name"])
```

```
            elif choice == "2":
                display(load_all())
            elif choice == "3":
                display(search_students(input(" Enter USN / name / branch: ")))
            elif choice == "4":
                usn = ask(" USN to update: ", validate_usn)
                field = input(" Field (name/branch/semester/cgpa): ").strip().lower()
                update_student(usn, **{field: input(" New value: ")})
                print("  Record updated.")
            elif choice == "5":
                delete_student(input(" USN to delete: "))
                print("  Record deleted.")
            elif choice == "6":
                statistics()
            elif choice == "0":
                print(" Goodbye!")
                break
            else:
                print("  Please choose a number from the menu.")
        except (ValueError, KeyError) as err:
            print("  Error:", str(err).strip("'\""))


def demo():
    """Scripted run that exercises every operation, including invalid input."""
    if os.path.exists(DATA_FILE):
        os.remove(DATA_FILE)
    print(">> Add three students")
    add_student("25BTRAD049", "sriram satvik karanam", "cse-aide", 3, 8.9)
    add_student("25BTRAD101", "Ananya Rao", "CSE-AIDE", 3, 9.2)
    add_student("25BTRCS045", "Rahul Menon", "CSE", 3, 7.6)
    display(load_all())
    print("\n>> Try bad data (each must be rejected)")
    for bad in [("25BTRAD049", "Duplicate Guy", "CSE", 3, 8),
                ("ABC", "Wrong Usn", "CSE", 3, 8), ("25BTRAD150", "Asha K", "CSE", 9, 8),
                ("25BTRAD151", "Asha K", "CSE", 3, 11.5), ("25BTRAD152", "R2D2", "CSE", 3, 8)]:
        try:
            add_student(*bad)
        except ValueError as err:
            print("  Rejected:", err)
    print("\n>> Search 'rao', then search by USN 25BTRCS045")
    display(search_students("rao"))
    display(search_students("25BTRCS045"))
    print("\n>> Update Rahul's CGPA to 8.1 and semester to 4")
    update_student("25BTRCS045", cgpa=8.1, semester=4)
    display(search_students("Rahul"))
    print("\n>> Delete 25BTRAD101, then try deleting it again")
    delete_student("25BTRAD101")
    try:
        delete_student("25BTRAD101")
    except KeyError as err:
        print("  Error:", str(err).strip("'\""))
    display(load_all())
    statistics()
    print("\n>> Raw contents of", DATA_FILE)
    print(open(DATA_FILE).read())


if __name__ == "__main__":
    demo() if "--demo" in sys.argv else menu()
```

4. Menu (Interactive Mode)

`python student_manager.py` shows the following menu and repeats until option 0 is chosen:

```
===== STUDENT DATA MANAGER =====
 1. Add student      4. Update student
 2. View all         5. Delete student
 3. Search           6. Statistics
 0. Exit
Choose an option:
```

5. Output - Scripted Demonstration

`python student_manager.py --demo` runs every operation, including invalid inputs that must be rejected:

```
>> Add three students
+-------------+----------------------+------------+-----+-------+
| USN         | NAME                 | BRANCH     | SEM | CGPA  |
```

```
+-------------+----------------------+------------+-----+-------+
| 25BTRAD049  | Sriram Satvik Karanam | CSE-AIDE   | 3   | 8.90  |
| 25BTRAD101  | Ananya Rao            | CSE-AIDE   | 3   | 9.20  |
| 25BTRCS045  | Rahul Menon           | CSE        | 3   | 7.60  |
+-------------+----------------------+------------+-----+-------+
  3 record(s)

>> Try bad data (each must be rejected)
  Rejected: Duplicate USN 25BTRAD049 - record already exists
  Rejected: USN must look like 25BTRAD049 (2 digits + letters + 3 digits)
  Rejected: Semester must be a whole number from 1 to 8
  Rejected: CGPA must be between 0 and 10
  Rejected: Name must be 2-50 characters (letters, spaces, . ' - only)

>> Search 'rao', then search by USN 25BTRCS045
+-------------+----------------------+------------+-----+-------+
| USN         | NAME                 | BRANCH     | SEM | CGPA  |
+-------------+----------------------+------------+-----+-------+
| 25BTRAD101  | Ananya Rao           | CSE-AIDE   | 3   | 9.20  |
+-------------+----------------------+------------+-----+-------+
  1 record(s)
+-------------+----------------------+------------+-----+-------+
| USN         | NAME                 | BRANCH     | SEM | CGPA  |
+-------------+----------------------+------------+-----+-------+
| 25BTRCS045  | Rahul Menon          | CSE        | 3   | 7.60  |
+-------------+----------------------+------------+-----+-------+
  1 record(s)

>> Update Rahul's CGPA to 8.1 and semester to 4
+-------------+----------------------+------------+-----+-------+
| USN         | NAME                 | BRANCH     | SEM | CGPA  |
+-------------+----------------------+------------+-----+-------+
| 25BTRCS045  | Rahul Menon          | CSE        | 4   | 8.10  |
+-------------+----------------------+------------+-----+-------+
  1 record(s)

>> Delete 25BTRAD101, then try deleting it again
  Error: No student found with USN 25BTRAD101
+-------------+----------------------+------------+-----+-------+
| USN         | NAME                 | BRANCH     | SEM | CGPA  |
+-------------+----------------------+------------+-----+-------+
| 25BTRAD049  | Sriram Satvik Karanam | CSE-AIDE   | 3   | 8.90  |
| 25BTRCS045  | Rahul Menon          | CSE        | 4   | 8.10  |
+-------------+----------------------+------------+-----+-------+
  2 record(s)
  Students: 2 | Average CGPA: 8.50 | Topper: Sriram Satvik Karanam (8.9)

>> Raw contents of students.txt
25BTRAD049|Sriram Satvik Karanam|CSE-AIDE|3|8.9
25BTRCS045|Rahul Menon|CSE|4|8.1
```

6. Testing and Verification (CT)

Test case	Input	Expected behaviour	Result
Store	Add 3 valid students	3 lines appended; table shows 3 records	**Pass**
Duplicate key	Add USN 25BTRAD049 again	Rejected - duplicate USN	**Pass**
Invalid USN	USN = ABC	Rejected - wrong format	**Pass**
Invalid semester	Semester = 9	Rejected - must be 1 to 8	**Pass**
Invalid CGPA	CGPA = 11.5	Rejected - must be 0 to 10	**Pass**
Invalid name	Name = R2D2	Rejected - digits not allowed	**Pass**
Retrieve by name	Search 'rao'	Only Ananya Rao is shown (partial, case-insensitive)	**Pass**
Retrieve by USN	Search 25BTRCS045	Only Rahul Menon is shown	**Pass**
Update	Rahul: CGPA 7.6 → 8.1, semester 3 → 4	Change visible on the next search and in the file	**Pass**

Test case	Input	Expected behaviour	Result
Delete + repeat delete	Delete 25BTRAD101 twice	First succeeds; second gives 'No student found'	**Pass**
Persistence	Read raw students.txt at the end	File content equals the final table (2 records)	**Pass**

Manual check of statistics: final CGPAs are 8.9 and 8.1, so the average is (8.9 + 8.1) / 2 = 8.50 and the topper is Sriram Satvik Karanam (8.9) - identical to the program output.

7. Real World Exposure (RWE) - college student-record management

Scenario. A college department must keep each student's USN, name, branch, semester and CGPA. Earlier this is done in paper registers or scattered spreadsheets, which leads to duplicate entries, spelling mistakes and long search times. The program replaces this with one validated, permanent record file.

Need in a college	How the program helps
Admission / enrolment	*Add* stores each new student once; the duplicate-USN check prevents double admission entries.
Quick look-up by mentors and office staff	*Search* by USN, by part of a name or by branch gives the record in a moment instead of turning pages.
Semester promotion and result updates	*Update* changes semester or CGPA at the end of each term without retyping the whole record.
Students who leave or transfer	*Delete* removes the record cleanly; the file is rewritten safely.
Data quality	Validation guarantees that every stored record has a correct USN format, a proper name and CGPA in range.
Reports for the HOD / exam cell	*Display* prints a sorted table, and *Statistics* gives total students, average CGPA and topper.
Data is not lost on exit	Records stay in `students.txt` - the same data is available the next time the program is started.

Limitations and future scope: A text file is ideal for small data (one class or department). For thousands of students with many users at the same time, the same functions can be re-implemented over a database such as SQLite or MySQL, which gives indexing, concurrent access and access control.

Conclusion: The program shows how file operations (append, read, rewrite) and validation together form a reliable record-management system that is simple, permanent and easy to maintain.

LAB EXERCISE 13

CSV-Based Payroll Management System

Read employee CSV • calculate gross salary & deductions • generate salary summaries

■ QUESTION - Lab Exercise 13

CSV-Based Payroll Management System: Develop a Python program to **read employee records from a CSV file**, **calculate gross salary and applicable deductions**, and **generate salary summaries**.

(CT) Analyse **payroll requirements**, design **calculation logic**, **process CSV records**, **validate data**, and **verify salary calculations**.

(RWE) Apply the program to a **sample organisation's payroll process** and **interpret the generated salary summaries**.

1. Aim

To read employee details from `employees.csv`, compute each employee's monthly **gross salary**, **deductions** (PF, professional tax, income tax) and **net pay**, reject invalid rows with reasons, and produce an on-screen payroll register, department-wise summary and an output file `salary_summary.csv`.

2. Critical Thinking - Payroll Requirements and Calculation Design

2.1 Input and output

	Details
Input - employees.csv	emp_id, name, department, designation, basic_salary, bonus (optional). All amounts are monthly, in Rs.
Output (screen)	Payroll register, totals, highest / lowest paid, department-wise summary, list of rejected rows.
Output (file)	salary_summary.csv with every salary component of each employee - ready for accounts / bank upload.

2.2 Salary structure and rules (simplified for the lab)

Component	Rule	Type
Basic	Taken from the CSV file	Earning
HRA (House Rent Allowance)	20% of Basic	Earning
DA (Dearness Allowance)	10% of Basic	Earning
Transport allowance	Fixed Rs. 1,600	Earning
Bonus	From CSV (0 if blank)	Earning
GROSS SALARY	**Basic + HRA + DA + Transport + Bonus**	**Total earnings**
PF (Provident Fund)	12% of Basic	Deduction
Professional Tax	Rs. 200 if Gross > Rs. 15,000, otherwise 0	Deduction
TDS (income tax)	Marginal slabs on monthly gross: up to 25,000 → 0%; 25,001-50,000 → 5%; 50,001-1,00,000 → 10%; above 1,00,000 → 20%	Deduction
NET PAY	**Gross - (PF + Professional Tax + TDS)**	**Take-home**

Marginal tax means each part of the salary is taxed at its own slab rate (like real income-tax slabs), so a small salary increase can never reduce take-home pay.

2.3 Algorithm

Step	Action
1	Open the CSV with `csv.DictReader`; stop with a message if the file is missing, empty or a required column is absent.
2	For every row: check that emp_id, name and department are not blank; emp_id is not repeated; salary and bonus are numbers; salary > 0 and values are not negative.
3	Valid row → calculate HRA, DA, gross, PF, professional tax, TDS, total deductions and net pay.
4	Invalid row → do not pay; record *row number* + *reason* in the rejected list.
5	Print the register; compute totals, average, highest and lowest net pay; group by department.
6	Write all calculated components to salary_summary.csv.

2.4 Design decisions

- **Decimal instead of float:** money must be exact. Python's `decimal` module with `ROUND_HALF_UP` avoids errors such as 0.1 + 0.2 = 0.30000000000000004.
- **Rules stored as constants** at the top (HRA_RATE, PF_RATE, TDS_SLABS ...) - when the policy changes only one line changes.
- **Calculation separated from file reading** (`calculate_salary()` vs `read_employees()`) so it can be tested alone.
- **Bad rows never stop the run** - one wrong entry should not block everyone's salary; it is reported instead.

3. Complete Program - payroll.py

```
"""
Lab Exercise 13 : CSV-Based Payroll Management System
Reads employees.csv, computes gross salary and deductions, writes salary_summary.csv.
Usage : python payroll.py [employees.csv]   |   python payroll.py --test
Input columns : emp_id, name, department, designation, basic_salary, bonus (optional)
All amounts are MONTHLY, in Rs.  Rules below are simplified for lab use.
"""
import csv
import os
import sys
from collections import defaultdict
from decimal import Decimal, ROUND_HALF_UP

# ------------------------------------------------------------ payroll rules
HRA_RATE = Decimal("0.20")        # House Rent Allowance  = 20% of basic
DA_RATE = Decimal("0.10")         # Dearness Allowance    = 10% of basic
TRANSPORT = Decimal("1600")       # fixed transport allowance
PF_RATE = Decimal("0.12")         # Provident Fund        = 12% of basic
PT_LIMIT, PT_AMOUNT = Decimal("15000"), Decimal("200")   # professional tax
# Monthly TDS slabs applied marginally: (upper limit of slab, rate)
TDS_SLABS = [(Decimal("25000"), Decimal("0")), (Decimal("50000"), Decimal("0.05")),
             (Decimal("100000"), Decimal("0.10")), (None, Decimal("0.20"))]
REQUIRED = ["emp_id", "name", "department", "designation", "basic_salary"]


def money(value):
    return Decimal(value).quantize(Decimal("0.01"), rounding=ROUND_HALF_UP)

def calc_tds(gross):
    """Marginal slab tax: each slice of gross is taxed at its own rate."""
    tax, lower = Decimal("0"), Decimal("0")
    for upper, rate in TDS_SLABS:
        top = gross if upper is None else min(gross, upper)
        if top > lower:
            tax += (top - lower) * rate
        if upper is None or gross <= upper:
            break
        lower = upper
    return money(tax)

def calculate_salary(basic, bonus=Decimal("0")):
    hra, da = money(basic * HRA_RATE), money(basic * DA_RATE)
    gross = money(basic + hra + da + TRANSPORT + bonus)
    pf = money(basic * PF_RATE)
    pt = PT_AMOUNT if gross > PT_LIMIT else Decimal("0")
    tds = calc_tds(gross)
    deductions = pf + pt + tds
    return {"basic": money(basic), "hra": hra, "da": da, "transport": TRANSPORT,
            "bonus": money(bonus), "gross": gross, "pf": pf, "pt": pt, "tds": tds,
            "deductions": money(deductions), "net": money(gross - deductions)}


# ------------------------------------------------------------ file reading
def to_amount(text, label):
    try:
        value = Decimal(text.strip())
    except Exception:
        raise ValueError(f"{label} '{text}' is not a number") from None
    if value < 0:
        raise ValueError(f"{label} cannot be negative")
    return value

def read_employees(path):
    """Return (valid_rows, rejected_rows). Raises for missing/empty/bad-header files."""
    if not os.path.exists(path):
        raise FileNotFoundError(f"CSV file not found: {path}")
    valid, rejected, seen = [], [], set()
    with open(path, newline="", encoding="utf-8-sig") as f:
        reader = csv.DictReader(f)
        if reader.fieldnames is None:
            raise ValueError("CSV file is empty")
        headers = [h.strip().lower() for h in reader.fieldnames]
        missing = [c for c in REQUIRED if c not in headers]
        if missing:
            raise ValueError("Missing column(s): " + ", ".join(missing))
        for row_no, raw in enumerate(reader, start=2):      # row 1 is the header
            row = {(k or "").strip().lower(): (v or "").strip() for k, v in raw.items()}
            try:
                blanks = [c for c in ("emp_id", "name", "department") if not row[c]]
                if blanks:
                    raise ValueError("missing " + ", ".join(blanks))
                if row["emp_id"] in seen:
```

```
                    raise ValueError(f"duplicate emp_id {row['emp_id']}")
                row["basic"] = to_amount(row["basic_salary"], "basic_salary")
                row["bonus"] = to_amount(row.get("bonus") or "0", "bonus")
                if row["basic"] == 0:
                    raise ValueError("basic_salary must be above zero")
                seen.add(row["emp_id"])
                valid.append(row)
            except ValueError as err:
                rejected.append((row_no, err))
    if not valid and not rejected:
        raise ValueError("CSV file has a header but no employee rows")
    return valid, rejected


# ---------------------------------------------------------- reporting
def process(path, out_path="salary_summary.csv"):
    employees, rejected = read_employees(path)
    results = []
    for e in employees:
        results.append({**e, **calculate_salary(e["basic"], e["bonus"])})

    print("=" * 99)
    print(f" PAYROLL REGISTER - {os.path.basename(path)}  (amounts in Rs. per month)")
    print("=" * 99)
    print(f" {'ID':<6}{'NAME':<18}{'DEPT':<12}{'BASIC':>10}{'GROSS':>12}"
          f"{'PF':>10}{'PT':>6}{'TDS':>10}{'DEDUCT.':>11}{'NET PAY':>11}")
    print("-" * 98)
    for r in results:
        print(f" {r['emp_id']:<6}{r['name']:<18}{r['department']:<12}{r['basic']:>10,.2f}"
              f"{r['gross']:>12,.2f}{r['pf']:>10,.2f}{r['pt']:>6,.0f}{r['tds']:>10,.2f}"
              f"{r['deductions']:>11,.2f}{r['net']:>11,.2f}")
    print("-" * 98)

    total = lambda key: sum((r[key] for r in results), Decimal("0"))
    print(f" Employees processed : {len(results)}    Rows rejected : {len(rejected)}")
    print(f" Total gross  : Rs. {total('gross'):>12,.2f}")
    print(f" Total deduct.: Rs. {total('deductions'):>12,.2f}")
    print(f" Total net pay: Rs. {total('net'):>12,.2f}   "
          f"(average Rs. {money(total('net') / len(results)):,.2f})")
    best = max(results, key=lambda r: r["net"])
    least = min(results, key=lambda r: r["net"])
    print(f" Highest net  : {best['name']} (Rs. {best['net']:,.2f})")
    print(f" Lowest net   : {least['name']} (Rs. {least['net']:,.2f})")

    dept = defaultdict(lambda: [0, Decimal("0"), Decimal("0")])
    for r in results:
        d = dept[r["department"]]
        d[0] += 1; d[1] += r["gross"]; d[2] += r["net"]
    print("\n DEPARTMENT-WISE SUMMARY")
    print(f" {'Department':<14}{'Staff':>6}{'Gross (Rs.)':>16}{'Net (Rs.)':>16}")
    for name, (count, gross, net) in sorted(dept.items()):
        print(f" {name:<14}{count:>6}{gross:>16,.2f}{net:>16,.2f}")

    if rejected:
        print("\n REJECTED ROWS (not paid - please correct the CSV)")
        for row_no, err in rejected:
            print(f"   Row {row_no}: {err}")

    cols = ["emp_id", "name", "department", "basic", "hra", "da", "transport", "bonus",
            "gross", "pf", "pt", "tds", "deductions", "net"]
    with open(out_path, "w", newline="", encoding="utf-8") as f:
        writer = csv.writer(f)
        writer.writerow(cols)
        for r in results:
            writer.writerow([r[c] for c in cols])
    print(f"\n Summary written to {out_path}")
    return results


def run_tests():
    """Hand-calculated check: basic 40,000 and no bonus."""
    s = calculate_salary(Decimal("40000"))
    assert s["hra"] == Decimal("8000.00") and s["da"] == Decimal("4000.00")
    assert s["gross"] == Decimal("53600.00")            # 40000+8000+4000+1600
    assert s["pf"] == Decimal("4800.00") and s["pt"] == Decimal("200")
    assert s["tds"] == Decimal("1610.00")               # 1250 (25k-50k @5%) + 360 (3.6k @10%)
    assert s["net"] == Decimal("46990.00")              # 53600 - 4800 - 200 - 1610
    print("TEST 1 passed : basic 40,000 -> gross 53,600.00, deductions 6,610.00, net 46,990.00")
    low = calculate_salary(Decimal("10000"))
    assert low["gross"] == Decimal("14600.00") and low["pt"] == 0 and low["tds"] == 0
    print("TEST 2 passed : low salary -> no professional tax, no TDS")
    for bad, label in (("abc", "text"), ("-5", "negative")):
        try:
            to_amount(bad, "basic_salary")
        except ValueError:
            print(f"TEST 3 passed : {label} salary rejected")
```

```
    try:
        read_employees("no_such_file.csv")
    except FileNotFoundError:
        print("TEST 4 passed : missing file handled")


if __name__ == "__main__":
    if "--test" in sys.argv:
        run_tests()
    else:
        file_path = sys.argv[1] if len(sys.argv) > 1 else "employees.csv"
        try:
            process(file_path)
        except (FileNotFoundError, ValueError) as err:
            print("[ERROR]", err)
```

4. Sample Input - employees.csv

Thirteen rows: nine are valid; four are deliberately wrong (negative salary, non-numeric salary, missing name, duplicate employee ID) to demonstrate validation.

```
emp_id,name,department,designation,basic_salary,bonus
E101,Ananya Rao,Finance,Accountant,32000,2000
E102,Rahul Menon,IT,Software Engineer,45000,3000
E103,Priya Sharma,HR,HR Executive,28000,0
E104,Karthik Iyer,IT,Senior Developer,70000,5000
E105,Meera Nair,Finance,Finance Manager,95000,8000
E106,Arjun Das,Operations,Supervisor,24000,1000
E107,Sneha Kulkarni,IT,Data Analyst,52000,2500
E108,Vikram Singh,Operations,Clerk,12000,0
E109,Divya Pillai,HR,HR Manager,80000,6000
E110,Rohan Gupta,IT,Intern,-15000,0
E111,Farah Khan,Finance,Auditor,abc,500
E112,,Operations,Driver,18000,0
E102,Rahul Duplicate,IT,Tester,30000,0
```

5. Output

5.1 Run: `python payroll.py employees.csv`

```
================================================================================================
 PAYROLL REGISTER - employees.csv  (amounts in Rs. per month)
================================================================================================
 ID    NAME             DEPT            BASIC       GROSS        PF    PT       TDS    DEDUCT.    NET PAY
------------------------------------------------------------------------------------------------
 E101  Ananya Rao       Finance      32,000.00   45,200.00  3,840.00   200  1,010.00   5,050.00  40,150.00
 E102  Rahul Menon      IT           45,000.00   63,100.00  5,400.00   200  2,560.00   8,160.00  54,940.00
 E103  Priya Sharma     HR           28,000.00   38,000.00  3,360.00   200    650.00   4,210.00  33,790.00
 E104  Karthik Iyer     IT           70,000.00   97,600.00  8,400.00   200  6,010.00  14,610.00  82,990.00
 E105  Meera Nair       Finance      95,000.00  133,100.00 11,400.00   200 12,870.00  24,470.00 108,630.00
 E106  Arjun Das        Operations   24,000.00   33,800.00  2,880.00   200    440.00   3,520.00  30,280.00
 E107  Sneha Kulkarni   IT           52,000.00   71,700.00  6,240.00   200  3,420.00   9,860.00  61,840.00
 E108  Vikram Singh     Operations   12,000.00   17,200.00  1,440.00   200      0.00   1,640.00  15,560.00
 E109  Divya Pillai     HR           80,000.00  111,600.00  9,600.00   200  8,570.00  18,370.00  93,230.00
------------------------------------------------------------------------------------------------
 Employees processed : 9    Rows rejected : 4
 Total gross  : Rs.   611,300.00
 Total deduct.: Rs.    89,890.00
 Total net pay: Rs.   521,410.00   (average Rs. 57,934.44)
 Highest net  : Meera Nair (Rs. 108,630.00)
 Lowest net   : Vikram Singh (Rs. 15,560.00)

 DEPARTMENT-WISE SUMMARY
 Department     Staff     Gross (Rs.)       Net (Rs.)
 Finance            2      178,300.00      148,780.00
 HR                 2      149,600.00      127,020.00
 IT                 3      232,400.00      199,770.00
 Operations         2       51,000.00       45,840.00

 REJECTED ROWS (not paid - please correct the CSV)
   Row 11: basic_salary cannot be negative
   Row 12: basic_salary 'abc' is not a number
   Row 13: missing name
   Row 14: duplicate emp_id E102

 Summary written to salary_summary.csv
```

5.2 First lines of the generated file salary_summary.csv

```
emp_id,name,department,basic,hra,da,transport,bonus,gross,pf,pt,tds,deductions,net
E101,Ananya Rao,Finance,32000.00,6400.00,3200.00,1600,2000.00,45200.00,3840.00,200,1010.00,5050.00,40150.00
E102,Rahul Menon,IT,45000.00,9000.00,4500.00,1600,3000.00,63100.00,5400.00,200,2560.00,8160.00,54940.00
```

5.3 Error handling

`python payroll.py nothere.csv` and `python payroll.py blank.csv` (empty file)

```
[ERROR] CSV file not found: nothere.csv
```

```
[ERROR] CSV file is empty
```

6. Verification of Salary Calculations (CT)

Hand calculation 1 - employee with Basic Rs. 40,000, no bonus (this is the automated test case):

Item	Calculation	Amount (Rs.)
HRA	20% of 40,000	8,000.00
DA	10% of 40,000	4,000.00
Gross	40,000 + 8,000 + 4,000 + 1,600 + 0	**53,600.00**
PF	12% of 40,000	4,800.00
Professional tax	Gross > 15,000	200.00
TDS	(50,000 - 25,000) x 5% = 1,250 + (53,600 - 50,000) x 10% = 360	1,610.00
Total deductions	4,800 + 200 + 1,610	6,610.00

Item	Calculation	Amount (Rs.)
Net pay	53,600 - 6,610	**46,990.00**

Hand calculation 2 - checking a row of the real output (E101 Ananya Rao, Basic 32,000, Bonus 2,000): HRA 6,400 + DA 3,200 + Transport 1,600 + Bonus 2,000 + Basic 32,000 = **45,200** gross. PF = 3,840; PT = 200; TDS = (45,200 - 25,000) x 5% = 1,010. Deductions = 5,050. Net = **40,150**. This equals the program output for E101.

Test	What is checked	Result
1	Basic 40,000 gives gross 53,600, deductions 6,610 and net 46,990 (hand-calculated values)	**Pass**
2	Low salary (Basic 10,000): no professional tax and no TDS	**Pass**
3	Text salary ('abc') and negative salary are rejected	**Pass**
4	Missing CSV file handled with a message	**Pass**
5	Cross-total: Gross total 611,300.00 - Deductions 89,890.00 = Net 521,410.00	**Pass**
6	Department-wise net: 148,780 + 127,020 + 199,770 + 45,840 = 521,410 = overall net total	**Pass**

Automatic test run - `python payroll.py --test`

```
TEST 1 passed : basic 40,000 -> gross 53,600.00, deductions 6,610.00, net 46,990.00
TEST 2 passed : low salary -> no professional tax, no TDS
TEST 3 passed : text salary rejected
TEST 3 passed : negative salary rejected
TEST 4 passed : missing file handled
```

7. Real World Exposure (RWE) - interpreting the salary summary

Scenario. A mid-sized organisation with Finance, IT, HR and Operations departments runs its monthly payroll from a CSV exported by the HR system. The program produces the following management information:

Observation from the generated summary	Interpretation / use
9 employees paid, **4 rows rejected** (rows 11-14)	Payroll is incomplete until HR corrects: a negative salary, a non-numeric salary, a missing name and a duplicate ID (possible double payment). Validation prevents wrong or duplicate payments.
Total gross Rs. 6,11,300; deductions Rs. 89,890; net Rs. 5,21,410	Organisation's monthly cash requirement for salaries is Rs. 5,21,410 net, plus employer obligations on the deducted amounts (PF, tax to be deposited with authorities).
Deductions are 14.7% of gross (89,890 / 6,11,300)	On average about 85% of gross reaches employees; useful for explaining take-home pay and for budgeting.
IT department: Gross Rs. 2,32,400 (about 38% of total) with 3 employees	IT is the largest cost centre - helps in departmental budgeting and manpower planning.
Highest net: Meera Nair Rs. 1,08,630; lowest: Vikram Singh Rs. 15,560	Shows the pay range (about 7 times). For the lowest salary, tax is 0.00 because gross (17,200) is below the 25,000 tax-free slab.
Average net pay Rs. 57,934.44	Benchmark for comparing individual pay and for planning increments.
salary_summary.csv generated	The file can be given to the accounts team, used for payslips, or uploaded to a bank for salary transfer.

Note: The PF, tax and allowance rules used here are simplified for learning. A real payroll system follows the current labour and income-tax laws and the organisation's policy, but the structure - read data, validate, calculate by rules, summarise - remains the same.

Conclusion: The program processes employee data from a CSV file, validates it, calculates gross salary, deductions and net pay accurately using Decimal arithmetic, and generates department-wise and organisation-wide summaries that support decision-making.

LAB EXERCISE 11 - 13

Overall Summary

What was achieved in these three lab exercises

	Lab 11 - Log Reader	Lab 12 - Student Manager	Lab 13 - Payroll (CSV)
File type	Text (.log)	Text with delimiter (.txt)	CSV (.csv) in and out
File operations	Read line by line	Append, read, safe rewrite	csv.DictReader, csv.writer
Key modules	re, os, collections	os, re	csv, decimal, collections
Validation	Missing / empty file	USN, name, branch, semester, CGPA, duplicates	Missing columns, blanks, non-numeric, negative, duplicate IDs
Verified by	Manual count + 4 tests	11 test cases + manual statistics	Hand calculations + 6 checks
Real-world use	System monitoring, troubleshooting	College record keeping	Organisation payroll and budgeting

How to run the programs

```
python log_analyzer.py app.log        # Lab 11  (python log_analyzer.py --test  for self-tests)
python student_manager.py             # Lab 12  (python student_manager.py --demo  for the scripted run)
python payroll.py employees.csv       # Lab 13  (python payroll.py --test  for self-tests)
```

Skills demonstrated: file reading and writing, exception handling, regular expressions, data validation, modular functions, exact decimal arithmetic, testing against hand-calculated results, and applying programs to real organisational problems (CT + RWE).